ASSIGNMENT-4-CO4

NAME:	**M.MANASA**
REG NO:	**192425189**
COURSE CODE:	**ITA0329**
COURSE:	**MOBILE COMPUTING**

PART I - ASSIGNMENT 4A

Task 1 - Architecture Review

Diagram 1. Android layered architecture

APPLICATIONS

User-facing apps and components (Activity, Window)

|

V

APPLICATION FRAMEWORK

Package, Location, Notification services

|

V

ANDROID RUNTIME (ART) + NATIVE LIBRARIES

DEX execution, memory/GC | graphics, media, database

|

V

HARDWARE ABSTRACTION LAYER (HAL)

Standard interfaces to device hardware

|

V

LINUX KERNEL

Scheduling, memory, drivers, networking, security

|

V

HARDWARE

CPU, RAM, display, camera, sensors, radios

1.1 Technical deficiencies

No.	Deficiency	Why it is a problem
1	Drivers and Hardware Abstraction Layer (HAL) are missing.	An engineer cannot identify how hardware devices such as camera, display, sensors and network interface are controlled.
2	Native libraries and system services are missing.	The note does not show where graphics, media, database, networking and other reusable OS functions are provided.
3	The kernel is described as running applications.	The kernel manages processes, memory, scheduling, devices, files and system calls. Applications normally run in user space.
4	Kernel type and interfaces are not stated.	Without the kernel structure and system-call boundary, the design cannot specify how user-space components request OS services.
5	Runtime is not defined.	The note does not say whether code is bytecode or native code, who loads it, or whether interpretation/JIT/AOT compilation is used.
6	Permission enforcement is vague.	It does not identify who grants permissions, when user consent is requested, or which

		OS component checks access.
7	Power saving has no trigger or mechanism.	There is no policy for screen-off, idle, low battery, CPU frequency, Doze or background work.
8	Application-to-OS communication is absent.	The IPC/API route, such as Binder on Android, is not shown, so the boundary between app and system services is unclear.

1.2 Repaired architecture - Android

Layer (top to bottom)	**Responsibility**	**Exposes to layer above**	**Depends on layer below**
Applications	User-facing apps and their components.	Activities, services, app APIs and UI.	Android application framework.
Application Framework	Activity Manager, Window Manager, Package Manager, Location and Notification services.	Java/Kotlin APIs and system-service interfaces.	Android Runtime and native system services.
Android Runtime (ART)	Loads and executes DEX bytecode; uses interpretation, JIT and profile-guided AOT; manages memory and garbage collection.	Execution environment and core libraries.	Linux process, memory and thread services; native libraries.
Native Libraries	Graphics, media, database, SSL, web rendering and other native functions.	Native APIs through framework/JNI.	HAL and Linux kernel interfaces.

Hardware Abstraction Layer (HAL)	Provides standard interfaces to vendor hardware implementations.	Camera, audio, sensor, Bluetooth and other hardware interfaces.	Device-specific drivers and kernel.
Linux Kernel	Process scheduling, virtual memory, device drivers, networking, file systems, security enforcement support and system calls.	System-call interface and device access.	Physical hardware.
Hardware	CPU, memory, display, camera, sensors, storage and radios.	Physical resources.	Not applicable.

Security location: Android permissions are declared by the application and granted according to install-time or runtime rules. Package Manager records permissions; framework services and kernel-level isolation/SELinux enforce access at relevant API and resource boundaries. Each app normally runs under a separate Linux UID and sandbox.

Power management location: Android power services and device-idle policies coordinate with the kernel. The kernel controls scheduling, suspend, wake sources and CPU frequency through platform drivers/governors. Doze and App Standby restrict eligible background activity.

1.2A Binder IPC Architecture and Project Treble HAL Decoupling

To resolve Deficiencies 1, 4, and 8 with engineering precision, modern Android separates application execution, system services, and vendor hardware implementations across strict process and IPC boundaries:

- **Binder Inter-Process Communication (IPC) Pipeline:** Applications run inside isolated Dalvik/ART user-space processes and cannot invoke privileged system services (such as ActivityManagerService or LocationManagerService running inside the system_server process) via direct function calls. Instead, Android uses the Binder kernel driver (/dev/binder) to perform zero-copy or single-copy cross-process transactions. AIDL (Android Interface Definition Language) generates Proxy (client-side) and Stub (server-side) marshalling wrappers that serialize arguments into Parcel objects, pass transaction codes through ioctl() system calls into kernel space, and dispatch calls onto the target service's Binder thread pool while automatically attaching the caller's Linux UID and PID for permission auditing.
- **Project Treble and Vendor HAL Isolation:** Prior to Android 8.0, vendor HAL shared libraries (.so) were loaded directly into framework processes, forcing device manufacturers to rewrite proprietary hardware code for every OS upgrade. Project Treble introduced the HIDL (HAL Interface Definition Language) and subsequently AIDL-based HAL architecture, isolating vendor HAL modules into standalone daemon processes (such as android.hardware.camera.provider) communicating with framework services over /dev/hwbinder or Binder IPC. This cleanly decouples the OS framework from Linux kernel and vendor driver updates.

1.3 Design defence

This is a precise architecture because it names each layer, assigns a bounded responsibility, identifies the interface exposed upward and states the dependency below. It places ART, the Linux kernel, permission enforcement and power management explicitly. The description can be checked against implementation evidence such as process information, system services, HAL interfaces and kernel behaviour. Unlike a marketing overview, it explains the path through which an application requests services and how the operating system controls resources.

Task 2 - Android vs iOS vs Windows

2.1 Requirement model

```
Network/modem -> Kernel network stack/driver -> OS push service ->
App wake/notification -> Runtime starts/resumes app callback ->
Validate message and store securely
```

Element	**Answer**
Input	Platform architecture, kernel family, runtime, framework, app isolation, IPC and background-execution rules.
Output	A comparison that predicts message delivery, security, responsiveness and battery impact.
Objective	Select a reliable design for secure background message reception on all three platforms.
Constraints	Screen may be off; app may be backgrounded or terminated; delivery must be secure and timely; battery use must be limited.
Decision	Use platform-specific push/background mechanisms behind a common application-level messaging design.
Information needed	Kernel and process model, system push service, runtime, permissions, sandbox, IPC, background limits and notification APIs.

2.2 Hypothesis

The background process and system push-notification model will affect message delivery most, because the operating system may suspend or terminate an application and only a privileged system service can reliably wake or notify it.

2.3 Comparison matrix

Feature	Android	iOS	Windows
Kernel family	Linux kernel with a modular monolithic kernel design.	Darwin core uses XNU, combining Mach and BSD components.	Windows NT hybrid kernel architecture.
Kernel interface	Linux system calls, device nodes and Binder-mediated system services.	Mach traps/ports and BSD system-call interfaces; public app APIs normally hide kernel details.	NT system calls and Win32/WinRT APIs; app developers normally use documented frameworks.
Runtime	ART executes DEX; interpretation, JIT and profile-guided AOT are used.	Objective-C runtime supports dynamic dispatch, messaging and runtime metadata; Swift/Objective-C code is compiled to native code.	WinRT is an API/runtime; packaged apps commonly use native platform code or .NET depending on app technology.
Framework	Android SDK: Activity, Service, BroadcastReceiver, ContentProvider and system services.	Cocoa Touch/UIKit, Foundation and other system frameworks.	WinUI/UWP and Windows App SDK APIs, depending on app model.
Isolation	Per-app Linux UID, sandbox, SELinux and permission checks.	Per-app sandbox, code signing, entitlements and privacy permissions.	AppContainer/package identity and capability declarations for applicable app models.
IPC	Binder is the principal framework IPC mechanism; intents and providers are higher-level	XPC and Mach ports are used for system and process communication.	COM/RPC and app-service mechanisms, depending on API and app model.

	mechanisms.		
Messaging consequence	Use FCM/high-priority messages only for urgent, permitted cases; Doze and background limits affect timing.	Use APNs; the OS controls background wake and delivery, so silent pushes are not guaranteed immediate execution.	Use Windows push notification services and permitted background tasks; policy and app lifecycle affect execution.

2.4 Incoming message trace - Android and iOS

Step	**Android trace**	**iOS trace**
1. Network event	Push packet reaches device modem/network stack.	Push packet reaches device network stack.
2. OS/kernel	Linux network driver and TCP/IP stack deliver traffic to system networking components.	Darwin networking components process the incoming traffic.
3. System service	Google Play services / FCM infrastructure receives and routes the push to the app or notification path.	Apple Push Notification service infrastructure and system push daemon route the notification.
4. Wake/launch	System may briefly wake the app for an eligible high-priority message; Doze restrictions still apply.	System may display a notification or grant limited background execution for an eligible push.
5. Runtime/app	ART process executes receiver/service or callback, validates message and stores it.	Native app process executes the permitted notification callback, validates and stores data.

2.4A Push Connection Multiplexing and Quotas Across Mobile OS Families

Maintaining separate persistent TCP/TLS sockets for every installed messaging application would prevent the baseband modem from entering low-power sleep states, rapidly depleting battery life. Consequently, all three platforms multiplex push delivery over a single OS-managed persistent connection:

- **Android FCM Multiplexing and App Standby Buckets:** Google Play Services maintains a single persistent TLS/TCP connection (traditionally on port 5228) with heartbeat intervals dynamically adapted to cellular NAT timeouts. Under Android 9+ App Standby Buckets (Active, Working Set, Frequent, Rare, Restricted), the OS enforces strict daily quotas on high-priority FCM wake-ups; if an application repeatedly triggers high-priority pushes without displaying user-visible notifications, Android demotes its bucket or deprioritizes subsequent pushes.
- **iOS apsd Daemon and Notification Service Extensions:** iOS routes all remote pushes through the system daemon (apsd) over a single encrypted HTTP/2 connection to APNs. For silent background pushes (content-available: 1), iOS enforces a strict power and thermal budget via the Duet Activity Scheduler (DAS), coalescing or deferring delivery when the device is on battery power or in Low Power Mode. Notification Service Extensions provide a bounded ~30-second execution window to decrypt end-to-end encrypted payloads before rendering alert banners.
- **Windows WNS and Background Task Infrastructure:** Windows Push Notification Services (WNS) uses the Windows Notification Platform system service to receive raw, toast, tile, or badge notifications, activating registered IBackgroundTask handlers subject to CPU and network resource quotas.

2.5 Changed condition - app terminated

Prediction: delivery differs because Android and iOS apply different rules to force-stopped, terminated and backgrounded applications. Android generally does not deliver normal app broadcasts to a force-stopped package until it is launched again; eligible FCM messages may start a process in some non-force-stopped cases. iOS can display a remote notification even when the app is not running, but background content processing is not guaranteed. The responsible elements are the OS push service, process lifecycle policy and app state. Therefore the application must treat push as a notification signal, fetch authoritative messages securely when execution is allowed, and not assume every push produces an immediate callback.

Task 3 - Runtime Environment Analysis

Diagram 3. Android application launch sequence

```
User taps app icon -> Launcher sends launch request -> Activity
Manager checks process -> Zygote forks app process -> ART initialises
and loads code -> Activity lifecycle callbacks -> Window and views
are prepared -> First frame rendered
```

3.1 Execution model

Platform	Delivered form and loader	When executable
Android (Dalvik/ART)	APK contains DEX bytecode. Package Manager installs it; ART loads DEX and runtime metadata.	DEX may be interpreted/JIT-compiled at run time and AOT-compiled using profiles; native instructions execute when compiled or interpreted.
iOS	App bundle contains signed native machine-code binaries and resources. dyld loads images and links required libraries.	Code is compiled before distribution; loader maps and links native code at launch.
Windows	Depends on app technology: native PE machine-code binary or .NET intermediate language for managed apps. OS loader/.NET runtime loads it.	Native code is executable after mapping; managed code is JIT-compiled or may use native compilation depending on deployment/runtime.

3.2 Android launch sequence

1. User taps the app icon. Launcher sends an activity launch request; the package must be installed and enabled.

2. Activity Manager checks package/activity information and process state. If the process does not exist, Android requests a new app process.

3. Zygote forks the app process. Common runtime setup is inherited, reducing the cost of creating a process from scratch.

4. ART initialises the application runtime, loads DEX classes and performs required verification/compilation. The app's Application object is created.

5. Activity instance is created and lifecycle callbacks such as onCreate() and onStart() execute. Required resources and views are prepared.

6. Window Manager and graphics system arrange the window and render the first frame. The screen becomes visible.

3.3 Runtime strategies

Topic	**Android ART**	**iOS Objective-C/Swift**	**Windows**
Execution	DEX bytecode; interpreter and JIT, with profile-guided AOT.	Native machine code; Objective-C runtime performs dynamic method lookup/dispatch; Swift uses native compilation.	Native code or managed .NET IL/JIT, depending on app model.
Compilation	JIT compiles frequently used methods while running; AOT compiles selected code ahead of time using profiles.	Main application code is compiled before installation/distribution.	Native apps are compiled ahead of time; managed .NET code may be JIT-compiled.
Memory	Tracing garbage	ARC uses reference	Native apps manage

	collection manages managed heap objects.	counting; cycles require weak/unowned references or cycle-breaking design.	memory according to language; .NET uses garbage collection.
Native access	JNI and Android NDK connect managed code to native libraries and OS APIs.	Objective-C/Swift frameworks and C/C++ interfaces reach native services.	Win32/WinRT APIs and P/Invoke/native interfaces expose OS services.
Responsiveness	GC pauses can briefly affect work; JIT improves frequently used code after warm-up.	ARC releases objects when reference counts reach zero; retain/release work occurs as references change.	Depends on native or managed runtime; managed GC can introduce pauses.
Portability	DEX offers a common app format across supported Android CPU architectures; runtime handles device-specific execution.	Binaries are built and signed for supported Apple architectures and platform versions.	Depends on target architecture and app framework; managed IL can improve portability.

Conclusion: For a low-memory device, no runtime is automatically best. A small native application with controlled allocations may have a predictable footprint. ART benefits from a managed heap and compilation optimisation but needs runtime/heap space. ARC avoids a tracing GC pause but reference-count metadata and retain cycles still matter. The best choice is the one with the lowest measured peak memory and acceptable responsiveness on the target device.

3.3A Zygote Copy-on-Write Memory Sharing and Concurrent Copying GC Internals

Two internal runtime mechanisms explain how Android achieves fast process creation and sub-millisecond GC pause times despite executing managed bytecode:

- **Zygote Preloading and Linux Copy-on-Write (CoW):** During system boot, the init process launches Zygote, which preloads thousands of core Android framework classes, drawable resources, and shared libraries into read-only memory pages before opening a listening domain socket. When ActivityManagerService requests a new application process, Zygote executes a Linux fork() system call. Thanks to kernel Copy-on-Write (CoW) virtual memory paging, the newly forked app process shares all preloaded framework pages with Zygote and other apps without duplicating physical RAM until the app explicitly writes to a page, drastically reducing both launch latency and aggregate Proportional Set Size (PSS).
- **ART Concurrent Copying (CC) Garbage Collector:** Earlier Dalvik runtimes suffered from multi-millisecond 'stop-the-world' mark-sweep garbage collection pauses that caused dropped UI frames (jank). Modern ART uses a Concurrent Copying (CC) generational garbage collector with read barriers. Short-lived UI allocations are reclaimed rapidly in the young generation space, while heap compaction moves live objects concurrently alongside running application threads, limiting pause times to under 1–2 milliseconds.

3.4 Cold and warm start

Step	Cold start	Warm start
Process creation	Required if no process exists.	Usually skipped if process remains alive.
Runtime initialisation	Required.	Already initialised; usually skipped.
Class loading	Initial required classes are loaded.	Previously loaded classes may be reused.
Activity creation	New Activity instance and callbacks.	A new Activity may still be created; if returning to an existing instance, lifecycle path differs.

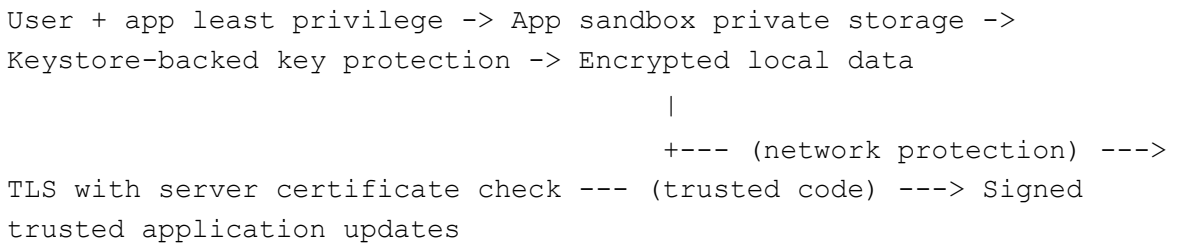
Rendering	First frame must be produced.	Frame is rendered using existing process/resources where retained.

3.5 Dependency analysis

The kernel must first create and schedule a process and provide memory and thread services. ART must initialise before managed application code can execute. The framework must then create the Activity and deliver lifecycle callbacks. Therefore framework callbacks cannot safely run before the process, memory, runtime and required system services exist. This ordering prevents application code from using unavailable execution or OS facilities.

Task 4 - Security Forensics

Diagram 4. SecureNotes repaired security design



4.1 Defects and bypassed mechanisms

Original decision	Mechanism bypassed	Risk
Request every permission at install time for future use.	Least privilege and runtime permission model.	Users may grant unnecessary access; modern Android permissions may require runtime consent and may be denied.
Save token in a file readable by other apps.	Application sandbox and secure storage.	Another app or a compromised device may steal the token and impersonate the user.
Accept updates from any source without verifying producer.	App signing and package integrity.	A malicious or modified package can replace trusted application code.
Use plain unencrypted server connections.	TLS transport protection and server authentication.	An attacker on the network can read or modify credentials and messages.

4.2 Attack scenario

1. A malicious app is installed on the same device.

2. Because the token is stored in a shared/readable file, the malicious app reads it. Confidentiality is first lost at this file-read step.

3. The attacker uses the copied token to authenticate to the SecureNotes server.

4. On public Wi-Fi, an attacker also intercepts the unencrypted connection and modifies a request. Integrity is lost when the server accepts the altered request.

5. An unverified update source can later deliver a modified application, making the compromise persistent.

4.3 Minimal repairs - Android

- Request only permissions needed for a specific feature and request dangerous permissions at the point of use. Handle denial gracefully.
- Store authentication tokens in app-private storage and protect sensitive keys using Android Keystore; encrypt sensitive data where appropriate.
- Accept updates only through a trusted distribution channel and verify package signing identity. Android package signature checks protect update continuity.
- Use HTTPS with TLS certificate validation. Do not allow cleartext traffic for the API.

These controls appear at different architecture boundaries: Package Manager and runtime permission checks, per-app Linux UID sandbox/SELinux, Keystore-backed key services, package-signature verification and the network/TLS stack.

4.3A Cryptographic Key Isolation (TEE/StrongBox) and Network Security Configuration

To ensure the minimal repairs in Section 4.3 resist both local privilege escalation and active network Man-in-the-Middle (MitM) attacks, Android enforces hardware-backed cryptographic boundaries and declarative transport policies:

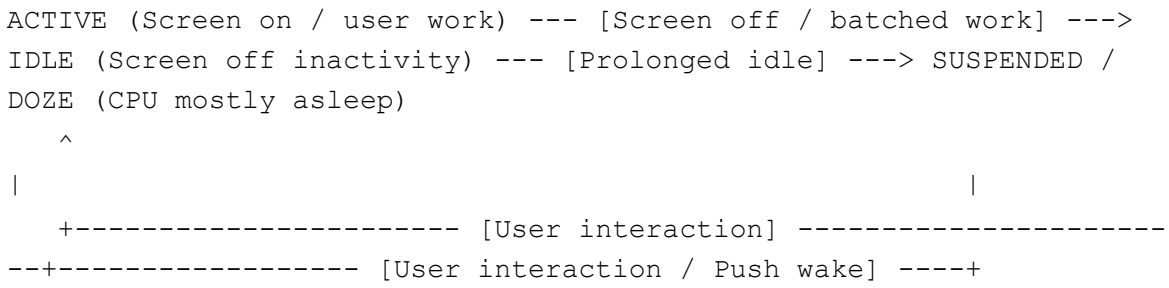
- **Hardware-Backed Keystore Isolation (TEE and StrongBox):** When SecureNotes generates an AES-256-GCM symmetric master key inside Android Keystore, raw key material never enters the application's process address space. Instead, cryptographic operations (encrypt/decrypt) are marshalled via IPC to the keymint/keymaster HAL executing inside a hardware-isolated Trusted Execution Environment (TEE, such as ARM TrustZone) or a dedicated tamper-resistant Secure Element (StrongBox). Even if an attacker compromises the Linux kernel, raw key bytes cannot be extracted from RAM.
- **Declarative Network Security Configuration and APK Signature Scheme v2/v3:** Android 9+ disables cleartext HTTP traffic by default, and apps should explicitly declare a res/xml/network_security_config.xml file setting cleartextTrafficPermitted='false', restricting trust anchors to system CAs (excluding user-installed proxy certificates in release builds), and optionally enforcing certificate pinning for critical authentication endpoints.

4.4 Regression check

Legitimate behaviour: the user may choose to attach a photo. The app requests CAMERA permission only when the camera feature is selected. If granted, the camera framework provides access; if denied, the user can still create and save a text note. The security repairs do not remove the authorised camera feature.

Task 5 - Power Management Reasoning

Diagram 5. Device power-state transitions



5.1 Power-state policy

State	Hardware/kernel behaviour	Entry condition	Exit condition	Allowed work
Active	CPU runs at suitable frequency; display and radios operate as needed; scheduler runs foreground work.	User interacts, sensor event requires immediate work, or an authorised wake event occurs.	Screen off and no immediate work remains.	Foreground app, short services and required system tasks.
Idle	Display off; CPU enters low-power idle; radios and sensors use batching; kernel reduces wake-ups.	Screen off and stationary/inactive for a defined period.	User interaction, authorised push event.	Limited maintenance, batched jobs and permitted push processing.
Suspended / Doze	CPU cores sleep; only selected wake sources remain; network and background execution are	Device remains unplugged and idle; no user interaction.	User interaction, charging or permitted high-priority wake event.	Kernel/system wake sources and narrowly permitted urgent work.

	restricted.			

5.2 Urgent push while suspended

The previous policy is insufficient if it assumes no work can run during suspension. Do not keep the processor awake continuously. Use a platform push service to signal urgent work and grant only a short, controlled wake period. The app validates the push, records the event and schedules longer processing through WorkManager/JobScheduler. Any wake lock must be short and released in a finally-style cleanup path.

5.2A Cellular Radio Resource Control (RRC) Tail Energy and Doze Maintenance Windows

Understanding why unbatched background network traffic drains battery requires examining cellular modem power states and Android's Doze maintenance cycle:

- **Cellular RRC Tail Energy Phenomenon:** In LTE and 5G cellular modems, transitioning from low-power RRC_IDLE to high-power RRC_CONNECTED requires signalling handshakes and radio amplifier ramp-up. After transmitting even a tiny 200-byte packet, the modem remains in a high-power tail state (RRC_CONNECTED / FACH) for 5 to 12 seconds waiting for potential follow-up packets before dropping back to idle. Sending one packet every 15 seconds keeps the cellular radio continuously locked in its highest-power state.
- **Doze Maintenance Windows and Wakeup Coalescing:** During deep Doze mode, Android defers standard alarms, background network access, and sync adapters, periodically opening brief Maintenance Windows (starting at 1 hour and exponentially backing off to every 6 hours) where all apps' batched jobs and pending network requests execute simultaneously in a single radio burst before returning to deep sleep.

5.3 Battery saver at 15%

Level	Change at 15% battery
Kernel/hardware	Reduce CPU performance ceiling or prefer efficiency cores; reduce display and background wake activity where platform policy allows.
Runtime/framework	Defer non-urgent jobs, reduce background scheduling and avoid unnecessary process work.
Application	Increase sensor sampling interval, batch uploads, reduce animations and postpone non-urgent refreshes; retain urgent safety events.
Trade-off	Location updates may be less frequent and some screens may refresh later. This is acceptable because critical alerts remain enabled while non-urgent work is delayed to preserve the shift.

Task 6 - RuralCare field-data collection design

Diagram 6. RuralCare offline capture and safe synchronisation

```
Worker enters patient form -> Validate + assign unique record ID ->
Encrypt and save local queue -> Network available?
   |-- [NO]  --> Keep encrypted queue
   |-- [YES] --> Send batch using TLS -> Server deduplicates by
record ID -> Acknowledgement received
```

Stage 1 - Problem model

Element	Specification
Input	Device class, patient form data, timestamps, user identity, battery, network state and sync acknowledgements.
Output	Protected local records and a server copy with no silent loss or duplication.
Objective	Complete a ten-hour shift offline-capable, securely and with reliable later synchronisation.
Constraints	Android/iOS/legacy feature phones; hours without network; limited battery; device loss; different storage/security capabilities.
Decisions	Platform strategy, local data format, encryption, sampling/power policy, sync protocol and legacy-phone limitations.

Stage 2 - Platform strategy

Use native Android and iOS applications for Classes A and B because they provide stronger platform security, encrypted storage options, background scheduling and modern network APIs. Use a restricted J2ME client for Class C where MIDP APIs and device storage permit, or a

minimal WML workflow for devices that only have a browser. Class C must collect the minimum necessary data and synchronise over HTTPS where supported. If the feature phone cannot protect patient data at rest or establish a secure connection, do not store identifiable patient records on it; use a privacy-minimised, short-lived workflow or exclude it from sensitive collection.

Stage 3 - Constraints and retained sync information

Device class	Runtime / framework	Security	Power controls	Offline storage	Connectivity
A - Android	ART; Android SDK/framework.	App sandbox, runtime permissions, Keystore, file/database encryption, TLS.	Doze, WorkManager, foreground service for justified tracking, sensor batching.	Encrypted SQLite/Room database; unique record IDs.	Wi-Fi, cellular; intermittent offline.
B - iPhone	Native compiled code; Cocoa Touch.	Sandbox, code signing, Keychain, Data Protection, TLS.	Background task APIs, location modes and OS scheduling.	Protected local database/files; Keychain for keys.	Wi-Fi/cellular; OS-controlled background execution.
C - J2ME/WML	JVM/MIDP for J2ME or browser/WML rendering.	Limited and device-dependent; HTTPS and PIN/access controls where available.	Very limited background execution; short sessions and low-frequency transfer.	RecordStore or minimal browser/session storage; avoid sensitive data if unencrypted.	Often low bandwidth; SMS/data or browser connection depending on device.

Retain the following for safe synchronisation:

- **Globally unique record ID:** prevents duplicate creation on retry.
- **Device ID and worker ID:** identifies the source and authorised owner.
- **Creation timestamp and local sequence number:** preserves ordering and helps resolve conflicts.
- **Schema/version number:** allows server and client to interpret older records.
- **Payload hash or integrity value:** detects corruption or accidental changes.
- **Sync status, retry count and last attempt time:** supports restart after interruption.
- **Server acknowledgement/receipt ID:** the client deletes or marks a record synced only after confirmed acknowledgement.

Stage 4 - Complete solution specification

Architecture by device class

- **Android:** UI -> domain validation -> repository -> encrypted local database -> sync worker. Android framework APIs call Keystore, network and power services; ART executes application code.
- **iOS:** UIKit/Cocoa Touch UI -> domain validation -> repository -> protected local database -> background URLSession/task. Keychain stores key material; OS controls background execution.
- **Class C:** simple form/menu -> validation -> minimal local RecordStore or session queue -> HTTPS sync. Avoid long-running background work and do not retain sensitive fields on devices without adequate protection.

Capture and offline storage

1. Authenticate the worker and check that required permissions are granted.

2. Validate each field locally; assign a UUID, worker/device ID, timestamp, sequence and schema version.

3. Encrypt the record before writing it to persistent storage. Keep encryption keys outside the database using Keystore/Keychain when supported.

4. Write the record and its pending-sync status in one local database transaction.

5. Show a saved-offline indicator; never require network access to finish a form.

Security and lost-device behaviour

- Use least-privilege permissions and role-based server access.
- Encrypt records at rest; use hardware-backed key storage where available.
- Use HTTPS/TLS with server certificate validation for every synchronisation.
- Lock the app after inactivity; require device authentication for sensitive screens.
- Support account revocation and remote wipe for managed smartphones. On loss, revoke the worker token and invalidate active sessions.
- On Class C devices without secure storage, collect only non-identifying reference data or do not permit patient-record caching.

Ten-hour power policy

Keep the screen off between forms, use sensor batching, avoid continuous partial wake locks, and schedule sync when connected to Wi-Fi or charging unless the queue reaches a defined urgent threshold. At low battery, reduce non-critical refreshes and batch uploads. Do not disable urgent safety communication.

Synchronisation procedure

1. Check network availability and authenticate using a short-lived token.

2. Read a bounded batch of pending records in creation order.

3. Send each record with its unique ID, schema version and integrity hash over TLS.

4. Server validates identity, schema and ID. If the ID already exists, server returns the existing acknowledgement rather than inserting a duplicate.

5. Server commits the record and returns an acknowledgement ID.

6. Client marks the record synced only after validating the acknowledgement.

7. If connection fails, retain unacknowledged records and retry with exponential back-off. Re-sending the same ID is safe because the server operation is idempotent.

8. Periodically reconcile server acknowledgements with the local queue. Delete local data only according to retention policy and confirmed server receipt.

Failure handling: no network -> keep encrypted queue; permission denied -> explain and disable only the dependent feature; storage full -> stop new sensitive capture and alert the worker; server error -> retain queue and retry later; app/process death -> recover from transactionally stored queue; invalid key -> stop access and escalate rather than writing plaintext.

Stage 4A - SQLCipher Database Encryption and Idempotent Sync Payload Specification

To ensure offline records on Android and iOS devices remain tamper-evident and cryptographically isolated even during prolonged disconnection:

- **Page-Level Database Encryption at Rest:** On Android, RuralCare wraps its Room persistence layer with SQLCipher, encrypting every 4096-byte database page using AES-256-CBC with per-page initialization vectors and HMAC-SHA512 page authentication. On iOS, the SQLite database file is created with

NSFileProtectionCompleteUntilFirstUserAuthentication (or NSFileProtectionComplete for active-screen access), binding file decryption keys to the device Secure Enclave.

- **Idempotent Sync Protocol and Conflict Resolution:** Each batch upload transmits a JSON/Protobuf envelope containing an Idempotency-Key header and per-record UUIDv4 identifiers. On the server side, a unique database constraint on (worker_id, record_uuid) guarantees that duplicate transmissions caused by dropped acknowledgements over rural 2G/3G links return HTTP 200 OK with the original receipt timestamp without creating duplicate clinical entries.

Stage 5 - Desk-check: Class A offline six hours, battery 20%, phone lost

Step	State/action	Result
1	Worker records forms offline; each receives a unique ID and is encrypted locally.	Records are saved in the protected queue.
2	Six hours pass without connectivity; sync worker cannot reach server.	Records remain pending; no data is lost locally.
3	Battery reaches 20%; low-battery policy reduces non-urgent sampling and defers routine sync.	Battery is conserved; queued records remain encrypted.
4	Phone is lost before sync.	No server acknowledgement exists for the pending records.
5	Organisation revokes worker token and triggers remote lock/wipe if device is managed and online.	Access is blocked when the device next receives the command; local data remains encrypted meanwhile.

Exposure: ideally no readable patient content is exposed because the database is encrypted and the key is protected by Keystore/device security. Loss: the unsynchronised records may be unavailable if the device cannot be recovered or the key is destroyed by wipe. Recoverable: records already acknowledged by the server are recoverable centrally; unacknowledged records are recoverable only if the device/key is recovered. This design cannot guarantee recovery of data that never reached the server.

Stage 6 - Attack your design

Adverse condition	**Prediction and trace**	**Refinement / result**
Stolen unlocked phone	Attacker may open the app while the session is active; sandbox alone does not stop use by someone holding an unlocked device.	Require app re-authentication for sensitive records, short inactivity lock, device lock and remote revocation. Risk reduced, not eliminated.
Sync interrupted mid-transfer	Some records may reach server but acknowledgements may be lost; client retries.	Idempotent unique IDs prevent duplicates; client marks only acknowledged records synced. Requirement met.
Class C device has no secure storage	Local patient data may be readable if stored in plaintext.	Do not cache identifiable records; minimise fields or exclude that device from sensitive workflows. This is a hard platform constraint.

Critical decision: never assume every platform has equivalent security. The Class C data-minimisation rule is essential. The design survives only when unsupported devices are prevented from storing sensitive information.

PART II - ASSIGNMENT 4B

Task 1 - Analyse and validate Android vs iOS

1.1 Architecture and process model

Area	Android	iOS
Layers	Linux kernel -> HAL -> native libraries -> ART -> application framework -> apps.	Darwin/Core OS -> Core Services -> Media -> Cocoa Touch -> apps.
Process creation	Zygote is the parent process used to fork app processes; Activity Manager coordinates launch.	launchd manages system services and process launch; app lifecycle is managed by UIKit/UIApplication.
IPC	Binder driver and Binder framework interfaces connect apps and system services.	Mach ports and XPC provide message-based inter-process communication.
Lifecycle	Activity: onCreate/onStart/onResume; Service and BroadcastReceiver have their own lifecycle rules.	UIApplication scene/app states and UIViewController lifecycle callbacks.

Android evidence commands (run on an emulator/device with adb):

```
adb shell getprop ro.build.version.release
adb shell ps -A | grep -E 'zygote|system_server|your.package.name'
adb shell dumpsys activity processes
adb shell dumpsys meminfo your.package.name
adb shell dumpsys package your.package.name
```

For iOS, use Xcode Instruments, Console and device logs to capture process launch, memory and lifecycle events. The assignment's Android adb commands do not apply to iOS.

1.2 Expose OS behaviour

Record process state, memory use and lifecycle logs at the same time. In Android, inspect oom_adj/process state in dumpsys output and compare PSS from dumpsys meminfo. A foreground process has higher importance; a cached/background process is more likely to be reclaimed under memory pressure. PSS estimates the process's proportional share of physical memory.

1.2A Android Low Memory Killer Daemon (lmkd) and oom_adj_score Hierarchy

Unlike desktop operating systems that rely on disk-backed swap partitions, mobile operating systems avoid flash wear and UI stalls by terminating lower-priority background processes when free RAM drops below watermarks:

- **Android oom_adj_score Process Priority Tiers:** Android's user-space Low Memory Killer Daemon (lmkd) monitors kernel memory pressure stalls (PSI - Pressure Stall Information) and terminates processes in descending order of their oom_adj_score: Cached Apps (score 900–999) are reclaimed first, followed by Previous Apps (700), Home App (600), Background Services (500), Perceptible/Foreground Services (200), Visible Apps (100), and Foreground Top App (0), while Persistent System Processes (-800 to -1000) remain protected.
- **iOS Jetsam Memory Enforcement:** On iOS, the Jetsam kernel subsystem performs an analogous role, enforcing hard per-process memory limits (High-Water Marks) and immediately terminating suspended or backgrounded apps with EXC_RESOURCE / JetsamEvent logs when system-wide physical pages fall low.

1.3 Three memory conditions

Test case	Prediction	What to record

Foreground	Process should remain alive unless severe system pressure occurs.	onResume log, PID, PSS, process state.
Background for 5 minutes	Process may remain cached; background work may be restricted.	onPause/onStop logs, PID and dumpsys activity processes.
Low-memory trim	System may call onTrimMemory; process may later be killed.	Trim callback, final lifecycle log, PID before/after and saved-state restoration.

1.4 Boundary test

Rotate the screen while a form dialog contains unsaved text and a background upload is running. This tests Activity recreation, state restoration and task ownership. Save the form state in a ViewModel or saved-state mechanism and keep upload work outside the Activity so rotation does not duplicate or cancel the upload.

Task 2 - Architecture assumptions vs measured reality

Implementation plan

Use the same deterministic workload for both solutions, such as parsing a fixed 5 MB route-data file or applying a pixel filter to a fixed image. Solution A runs the work on the main thread. Solution B runs it on a bounded ExecutorService or coroutine dispatcher and returns results to the UI thread.

Measurement	How to capture	Meaning
Task duration	System.nanoTime() around workload.	Elapsed processing time; compare repeated runs.
CPU/thread activity	Android Studio CPU Profiler or Perfetto; adb shell top -H.	Shows active threads and CPU utilisation.
UI quality	Frame timeline / jank frames / ANR logs.	Shows whether main-thread work blocks rendering.
Memory	dumpsys meminfo package; Android Studio Memory Profiler.	PSS, heap and allocation cost.
Predictability	Run at least 5 trials and compare median and worst result.	Worst-case behaviour matters for tracking deadlines.

2.1 Normal case

Expected: a large independent compute task moved off the main thread should keep the UI responsive. It may reduce perceived latency and jank even if total CPU work is similar. Record duration, frame timing and thread trace for both designs.

2.2 Counter-example

Use a very small task repeated only a few times. In the multi-threaded version, create a new thread for each tiny operation and wait for all threads. Thread creation, scheduling and synchronisation overhead may exceed the useful work, so the multi-threaded result can be slower and use more memory.

2.3 Engineering comparison

Metric	Single-threaded	Multi-threaded
CPU time	Record profiler CPU time.	Record profiler CPU time and context switches.
Thread count	Usually one worker/main thread for the task.	Record worker count and any thread creation.
Memory	Record heap/PSS before and after.	Record thread-stack and allocation overhead.
UI latency	Measure frame drops and response time.	Measure frame drops and response time.
Worst-case	Repeat under CPU load and record maximum.	Repeat under CPU load and record maximum; contention can increase variance.

2.3A VSYNC Frame Budget (16.6 ms / 8.3 ms) and ANR Watchdog Thresholds

To quantify UI responsiveness objectively during benchmark testing, engineers evaluate thread blocking against hardware display refresh deadlines and OS watchdog timers:

- **VSYNC Frame Rendering Deadlines:** On a standard 60 Hz mobile display, the UI thread and RenderThread must complete input handling, layout, measure, draw, and GPU command submission within a 16.67 ms VSYNC frame budget (or 8.33 ms on 120 Hz ProMotion/LTPO displays). Any main-thread operation exceeding this budget causes a missed VSYNC deadline, resulting in visible frame stutter (jank).

- **ANR Watchdog Timeouts and Thread Stack Cost:** If the main thread remains blocked and fails to process an input event within 5 seconds (or a BroadcastReceiver within 10 seconds), Android's ActivityManager triggers an Application Not Responding (ANR) dialog and writes thread stack traces to /data/anr/traces.txt.

2.4 Full-day battery requirement

Prefer bounded worker pools, batching and deferred WorkManager jobs for non-urgent work. Where the platform exposes scheduling/affinity controls, efficiency cores may be preferred for background work, but ordinary applications should not assume they can directly pin threads to a particular core. Reduce unnecessary wake-ups and measure energy using Battery Historian or the Energy Profiler.

Task 3 - Runtime behaviour under changing execution

Stage A - Workloads and measurements

- **CPU-bound:** run a repeatable sorting or numeric loop with fixed input.
- **Allocation-heavy:** create and release a large number of short-lived objects.
- **UI-list:** render and scroll a fixed list with the same row layout.

Capture cold/warm start time, compilation state, GC events and heap growth. Example commands:

```
adb shell dumpsys package your.package.name
adb shell cmd package compile -m verify -f your.package.name
adb shell cmd package compile -m speed-profile -f your.package.name
adb logcat -d | grep -i 'GC'
adb shell dumpsys meminfo your.package.name
```

Stage B - Compilation mode predictions

Mode	Prediction	Workload most affected
verify/interpret-oriented	Less ahead-of-time optimisation; lower compiled-code footprint may be possible, but execution can be slower.	CPU-bound loop.
speed-profile	Frequently used code is optimised using profile information; startup and hot paths may improve after profile collection.	Repeated CPU-bound and common UI paths.
speed	More code compiled for speed; may increase storage/compile work while improving steady-state execution.	Long-running compute workload.

Baseline Profiles identify important code paths so they can be compiled/optimised earlier. JIT adapts while the app runs; profile-guided AOT uses observed hot paths. Record actual results because device, OS version and compilation state change the outcome.

Stage C - Production failures

Memory leak test: retain an Activity in a static field or grow a cache without eviction. Observe heap growth across repeated screen opens. A garbage collector cannot reclaim objects that remain reachable. Use Android Studio Memory Profiler or LeakCanary to identify retained objects. Main-thread blocking test: perform a deliberate long sleep or disk/network operation on the UI thread in a test build; StrictMode or ANR traces can reveal the violation. Remove these test defects before release.

Stage C1 - Cloud Profiles, Dex2oat Background Compilation, and Retention Graph Analysis

Modern Android deployments bridge the gap between fast initial installation and optimal steady-state runtime speed through cloud-distributed compilation profiles and automated heap diagnostics:

- **Cloud Profiles and Idle-Time dex2oat Compilation:** Instead of compiling an entire APK ahead of time during installation (which delays app availability and inflates .odex storage), ART records hot methods and classes into a local profile file during initial runs. When the device is idle and charging, the background dex2oat daemon compiles only the profiled hot methods into native ELF machine code (.odex/.vdex). Google Play Cloud Profiles aggregate anonymized startup profiles from early users and bundle them directly into the APK installation metadata so new users benefit from AOT-compiled startup paths immediately.

- **GC Root Shortest-Path Leak Diagnostics:** When LeakCanary or Android Studio Memory Profiler detects an Activity instance still allocated after onDestroy(), it triggers an HPROF heap dump and computes the shortest strong reference path from a GC Root (such as a static singleton, active thread, or JNI global reference) to the leaked Activity, pinpointing the exact reference preventing garbage collection.

Stage D - Cross-platform runtime

Runtime	**Execution / memory**	**UI and native API access**
Android ART	DEX with JIT/AOT and tracing GC.	Android UI framework; JNI/NDK for native APIs.
Swift + ARC	Native compiled code; reference counting.	UIKit/SwiftUI; direct Apple frameworks; ARC release work occurs as references change.
Flutter	Dart AOT in release builds; Dart garbage collection; Flutter rendering engine.	Consistent widget rendering; platform channels/plugins for native APIs.
React Native / WebView hybrid	JavaScript engine; React Native communicates with native modules, while WebView runs web content in a browser engine.	Bridge/JSI or WebView interfaces; extra coordination can affect startup and memory.

Recommendation: for a product requiring Android and iOS with consistent UI and a shared codebase, Flutter is a reasonable candidate if the team's benchmark confirms acceptable startup, memory and native-plugin performance. If deep platform integration and minimum overhead dominate, separate native apps may be better. The final choice must be tied to the measurements collected in Stages A-C.

Task 4 - Secure mobile application audit

4.1 STRIDE threat model

Threat/asset	Attack	Protection	Residual risk
Spoofing - login token	Steal or replay token.	Keystore/Keychain-backed secrets, short-lived tokens and server-side revocation.	Compromised unlocked device may still use an active session.
Tampering - local database	Modify appointment records.	Encrypted database, integrity checks and server validation.	Rooted/jailbroken device may bypass app controls.
Repudiation - audit history	User denies an action.	Server-side authenticated audit events with timestamps.	Compromised server or account can affect trust.
Information disclosure - patient data	Read app files from another app.	Sandbox/UID separation, encryption at rest and access control.	OS compromise or unlocked-device access remains a risk.
Denial of service - API	Flood endpoint or exhaust app resources.	Rate limiting, bounded retries and input limits.	Large-scale distributed attacks remain possible.
Elevation of privilege - exported component	Malicious app invokes an unintended component.	Set components non-exported unless needed; validate caller/input and use permissions.	Incorrect exported configuration can expose functions.

4.2 Asset decision trace

Asset	Attack vector	Mechanism	Result

Auth token	Read local files / replay token	Private storage, Keystore, expiry and revocation	Partially blocked
Local database	Other app reads or alters data	Sandbox, encryption, database integrity checks	Partially blocked
API traffic	Network interception	TLS certificate validation	Blocked for ordinary network interception
Exported activity	Malicious app launches internal screen	Non-exported component and input validation	Blocked if configured correctly

4.3 Weaken one control

Prediction: set an internal Activity to exported true without an access permission. Another application may be able to launch it using an explicit Intent. The affected asset is the internal screen and any data/action it exposes. The audit verdict should change from Blocked to Exposed or Partially Blocked. Confirm with a controlled test app and adb shell am start; record the manifest and result.

4.4 Third-party library incident

The leaked asset is the device identifier and the affected layer is the application/network boundary. Sandbox and permissions do not prevent an app or its library from sending data through its own network permission. Detect it through static dependency/code analysis, runtime network hooks and packet capture. Remove or update the library, prevent identifier collection unless essential, and enforce TLS with endpoint allow-listing and server-side validation. TLS protects the channel but does not stop a trusted library from sending data to the wrong destination, so data minimisation and endpoint control are also required.

4.4A Intent Redirection, PendingIntent Mutability, and Play Integrity Attestation

In real-world mobile security audits, subtle Inter-Process Communication (IPC) flaws and device integrity checks frequently determine whether an attacker can bypass sandbox boundaries:

- **Intent Redirection and Immutable PendingIntents:** If an exported component accepts an embedded nested Intent from an untrusted caller and passes it directly to startActivity() or sendBroadcast(), a malicious app can abuse the victim app's higher privileges or internal non-exported components (Intent Redirection). Similarly, passing a mutable PendingIntent (FLAG_MUTABLE) to an external notification or third-party app allows the recipient to alter the underlying Intent action or target. Audits must enforce FLAG_IMMUTABLE and strict component allow-listing.
- **Hardware Key Attestation and Play Integrity Verification:** To detect rooted devices, custom ROMs, or tampered APK signatures before issuing sensitive session tokens, mobile applications integrate the Google Play Integrity API (or Apple App Attest), which provides cryptographically signed hardware-backed verdicts confirming genuine app binary integrity and verified device boot state.

4.5 Biometric and enterprise extension

Biometric login changes the threat model by adding a user-authentication factor. Bind access to cryptographic keys that require device authentication; do not store biometric templates in the app. Verify key properties/attestation where supported. An enterprise work profile creates a separate trust boundary: work data, keys and apps must remain separated from personal data. Audit logic must check profile ownership, managed configuration, remote wipe and policy enforcement. A remote wipe should target enterprise data rather than personal data where the platform supports selective wipe.

Task 5 - Three power engineers, one device

Technique	Trigger/maintained state	Expected power effect	Side effect / evidence
DVFS and core selection	CPU load, thermal state and performance demand; frequency/voltage and core utilisation.	Lower frequency/voltage and efficiency-core use reduce CPU energy for suitable workloads.	May increase processing time; record frequency/core traces and CPU energy estimate.
Doze / App Standby	Screen off, stationary, unplugged; idle state and app bucket.	Reduces background network, alarms and job wake-ups.	Delayed background updates; record batterystats and job/alarm activity.
App-level batching	Queued records, job constraints, sensor batch and wakelock duration.	Combines work and radio use; fewer wake-ups and less repeated network setup.	Data may arrive later; record wake-lock time, radio-active time and batch size.

Conflict test: hold a partial wakelock continuously while the phone is idle. Prediction: the device may not enter the deepest idle state or may receive less benefit from Doze. Another conflict is frequent exact alarms that repeatedly wake the device and defeat batching. This demonstrates that an application can undermine OS-level power saving.

5.1 Heterogeneous big.LITTLE SoC Scheduling and Energy-Aware Scheduling (EAS)

Modern mobile System-on-Chips (SoCs) combine high-performance 'big' CPU cores with low-power 'LITTLE' efficiency cores under Linux Energy-Aware Scheduling (EAS). Dynamic power consumption scales proportionally to capacitance, voltage squared, and clock frequency ($P_{dynamic} = C \times V^2 \times f$). Because higher frequencies require higher operating voltages, running a background sync thread on a LITTLE efficiency core at 1.2 GHz consumes a fraction of the milliwatts required by a big core at 2.8 GHz. Furthermore, hardware Sensor Hubs (low-power

microcontrollers) batch accelerometer and GPS samples into hardware FIFOs while the main application processor remains in deep suspend, waking the CPU only when the FIFO reaches its batch watermark.

Deployment	**Recommended emphasis**	**Reason**
A. Always-on tracking	Sensor batching plus carefully justified foreground service and efficient-core scheduling.	Tracking needs continuity, but batched sensor delivery and controlled wake-ups reduce energy.
B. Gaming/media	DVFS and thermal-aware scheduling.	Sustained CPU/GPU load dominates; OS must balance performance, heat and battery.
C. Managed fleet	OS-level idle policy plus centrally configured app batching.	Mixed OS versions require robust platform scheduling and policy controls; fleet management helps standardise configuration.

Task 6 - Battery-aware secure field-tracking prototype

6.1 Problem model

Element	Specification
Input	Battery percentage and charging, network type, motion state, last upload time, queue size, device temperature and urgency.
Output	Sampling interval, upload/defer decision, encrypted queue and energy estimate.
Objective	Maximise useful tracking records delivered per unit of battery while protecting data.
Constraints	Full shift, variable network, temperature, battery, privacy and emergency delivery requirements.

Battery percentage alone is insufficient: two phones at 20% may need different actions. A phone on Wi-Fi and charging can upload immediately; a hot phone on LTE with a large queue may need to defer routine upload and reduce sampling.

6.2 State policy

Battery state	Network	Motion	Sampling interval	Upload action
High (>50%)	Wi-Fi	Moving	5-10 s	Batch upload every 1-2 min.
Medium (20-50%)	LTE	Moving	15-30 s	Upload bounded batches when queue threshold reached.
Low (≤20%)	LTE	Stationary	120 s	Defer routine

				upload until Wi-Fi/charging; preserve urgent records.
Any	None	Any	30-120 s based on urgency	Queue encrypted records; retry with back-off.
Any	Wi-Fi + charging	Any	Normal interval	Upload pending batch immediately.
Any	Any; device hot	Any	Reduce non-critical sampling	Defer heavy processing and upload unless urgent.

Default state: normal battery, network unknown, motion unknown; start with a conservative sampling interval until state is measured. On every state transition, encrypt each record before queueing and require authenticated TLS upload. Architecture: UI on main thread; policy/domain logic on a bounded executor; sensor collection in an authorised foreground service where continuous tracking is justified; durable queue in encrypted database; WorkManager handles deferrable sync.

6.3 Prototype decision table - six conditions

Condition	**Policy**	**Security**	**Energy estimate**
1. Charging + Wi-Fi + moving	Sample 5 s; upload batch every 60 s.	AES-GCM before local storage; TLS upload.	Estimate from measured device baseline.
2. 80% + LTE + moving	Sample 10 s; upload every 3 min.	AES-GCM + TLS.	Measure CPU/radio active time.

3. 20% + LTE + stationary	Sample 120 s; defer routine upload.	Encrypted queue; urgent data sent through authenticated channel.	Lower wake-up/radio cost.
4. No connectivity	Sample 30-60 s; queue and retry with back-off.	Encrypt before queueing.	No radio upload cost; sensor/CPU cost remains.
5. Low-power mode	Sample 60-120 s unless urgent; defer batch sync.	Encryption remains enabled.	Reduced CPU wake-ups.
6. Device hot	Reduce non-critical sampling and heavy processing.	No security relaxation.	Lower CPU load and thermal stress.

For a simulation, define an energy estimate per cycle using measured or documented component costs: E_cycle = E_sensor + E_CPU + E_radio + E_display. Count the number of wake-ups and radio activations. Do not invent mAh values; calibrate with Battery Historian or a documented power model.

6.3A Finite-State Machine Policy Engine and Thermal Throttling Hooks

To prevent policy oscillation when battery voltage or signal strength fluctuates near transition thresholds, the prototype implements a deterministic Finite-State Machine (FSM) driven by Android system broadcasts and callbacks:

- **Multi-Signal Context Fusion and Hysteresis:** The engine subscribes to BatteryManager (EXTRA_LEVEL, EXTRA_PLUGGED), ConnectivityManager.NetworkCallback (distinguishing NET_CAPABILITY_NOT_METERED Wi-Fi from metered cellular links), ActivityRecognitionClient (detecting STILL vs. IN_VEHICLE/WALKING states to suppress redundant GPS fixes while stationary), and

PowerManager.OnThermalStatusChangedListener (scaling back batch compression and sampling intervals when thermal status reaches THERMAL_STATUS_MODERATE or SEVERE).

6.4 Validation

Choose three cases and calculate the expected policy by hand. Example: (1) charging + Wi-Fi -> normal sampling and immediate batch sync; (2) 20% + LTE + stationary -> 120-second sampling and defer routine upload; (3) no network -> encrypted local queue and retry later. Compare each expected action with prototype output. Matching results increase confidence that state transitions and policy rules are implemented correctly, although broader device testing is still required.

6.5 Scalability and load

Load	**Policy evaluations**	**Wake-ups**	**Memory/battery observation**
100 records	Record actual count	Record actual count	Record queue memory and energy estimate.
1,000 records	Record actual count	Record actual count	Check batch processing and memory growth.
10,000 records	Record actual count	Record actual count	Check bounded memory, database indexing and upload batching.

Batching reduces repeated radio wake-ups because one radio activation can transfer many records and amortise connection/setup costs. However, high-frequency tracking still consumes energy through repeated sensor sampling, CPU processing, storage writes and the amount of data

produced. Batching reduces communication overhead, not the underlying cost of continuous sensing.

6.6 Emergency requirement change

The current policy becomes unsuitable because it defers uploads and reduces sampling to conserve battery. The changed property is the priority/latency requirement: emergency delivery within seconds is more important than energy efficiency. Investigate an adaptive priority policy, a justified foreground tracking service, high-priority authenticated alert delivery and server-side fleet coordination. The accepted trade-offs are higher energy use, heat, network cost and greater location-privacy exposure. Apply the emergency mode only for authorised incidents and log its activation.

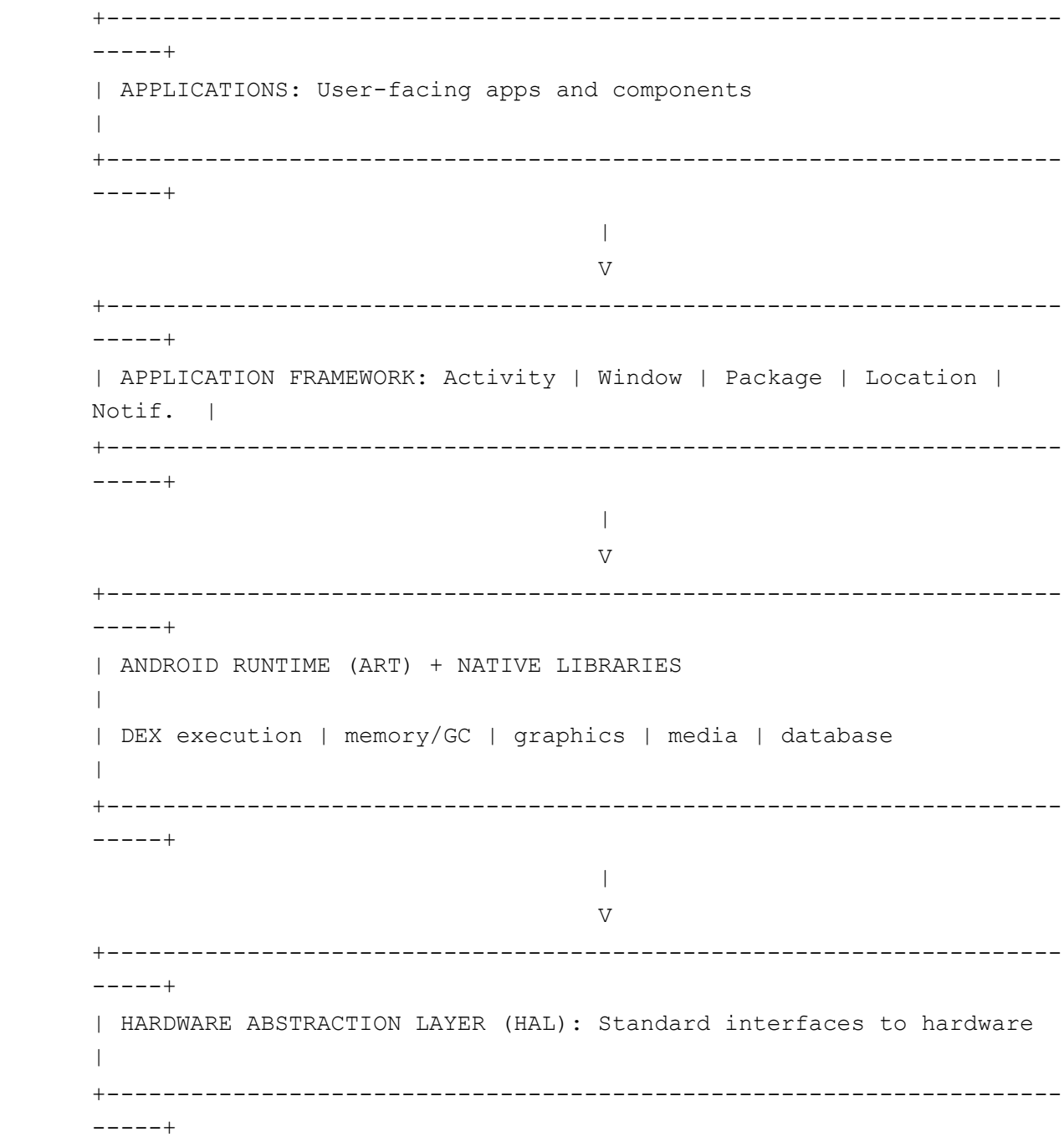
6.7 Technical reflection (under 150 words)

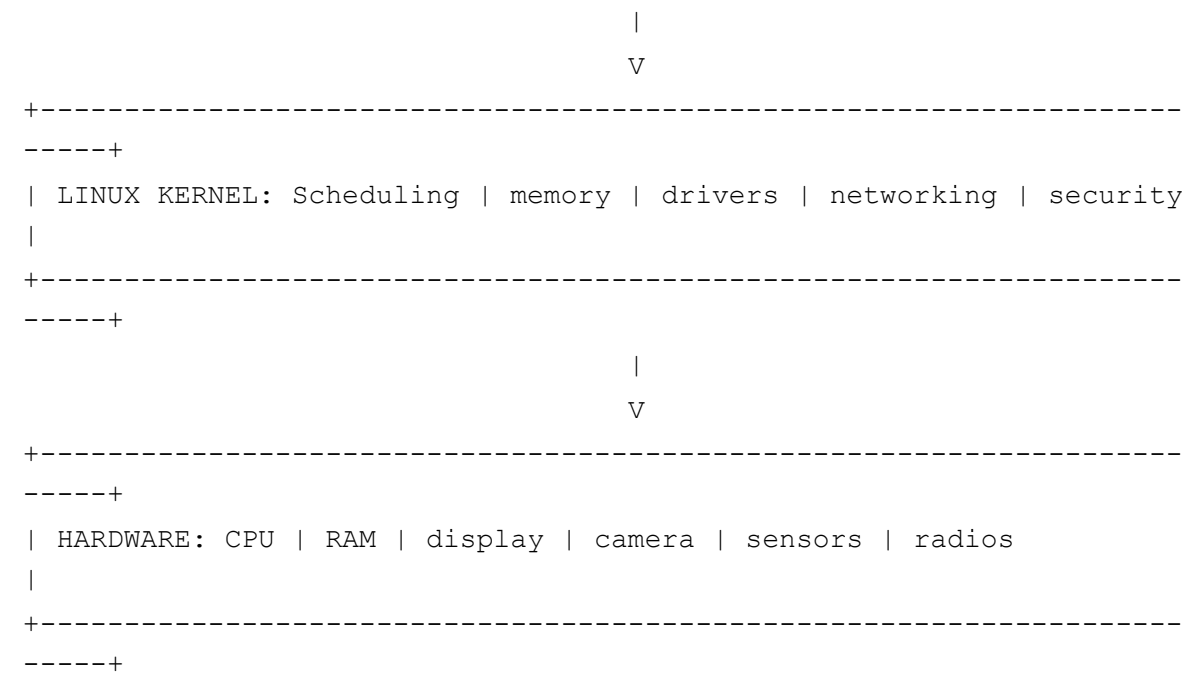
A power-saving technique depends on the device state and application requirement. Batching is useful when records can wait, because it reduces repeated CPU and radio wake-ups. It is unsuitable when an emergency requires location updates within seconds. Doze saves energy for idle applications, but it can delay routine background work. DVFS reduces CPU power, but may slow a compute-heavy task. Therefore no single OS or power feature is optimal for every application. The correct policy must consider battery, network, motion, temperature, urgency and data volume together. Our prototype should select a policy for each state and validate it against both energy use and delivery delay. A good design balances these measures rather than maximising only battery life.

DIAGRAMS - ARCHITECTURE AND WORKFLOWS

The following diagrams support the written solutions. They are simplified design views and should be read together with the detailed explanations and tables in each task.

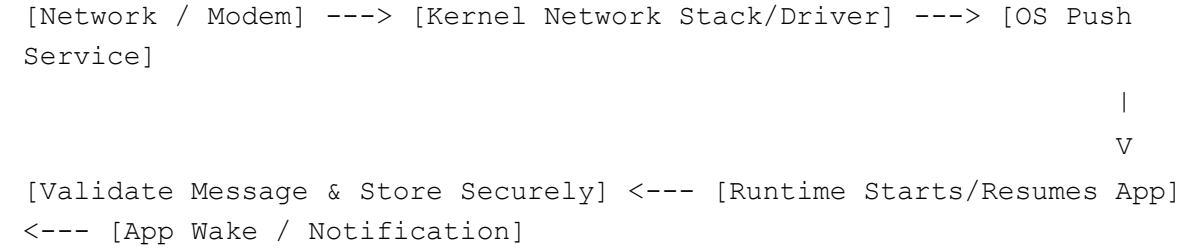
Diagram 1. Android layered architecture





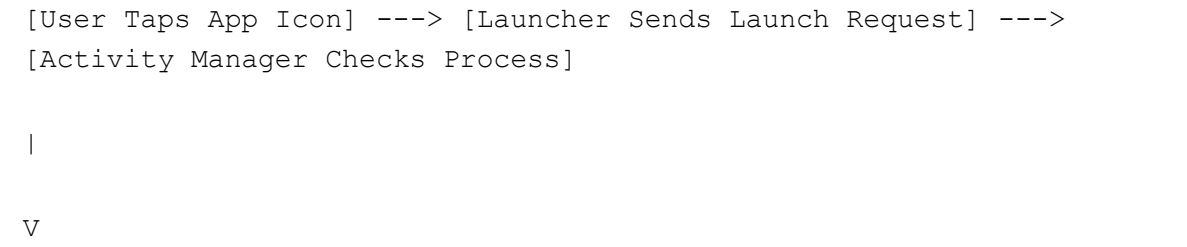
Related to Part I, Task 1: Android layers and the location of ART, HAL and kernel.

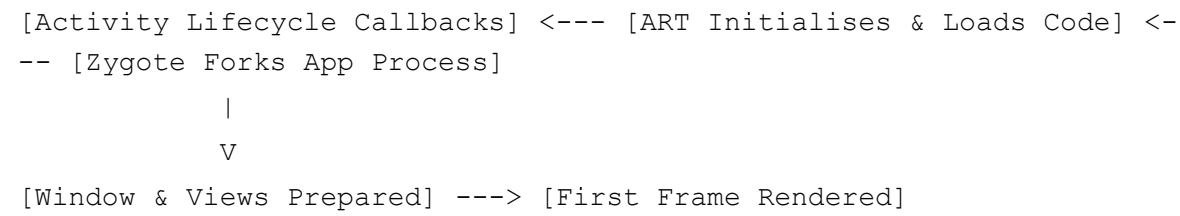
Diagram 2. Incoming push-message flow (simplified)



Related to Part I, Task 2: simplified path of an incoming message while the screen is off.

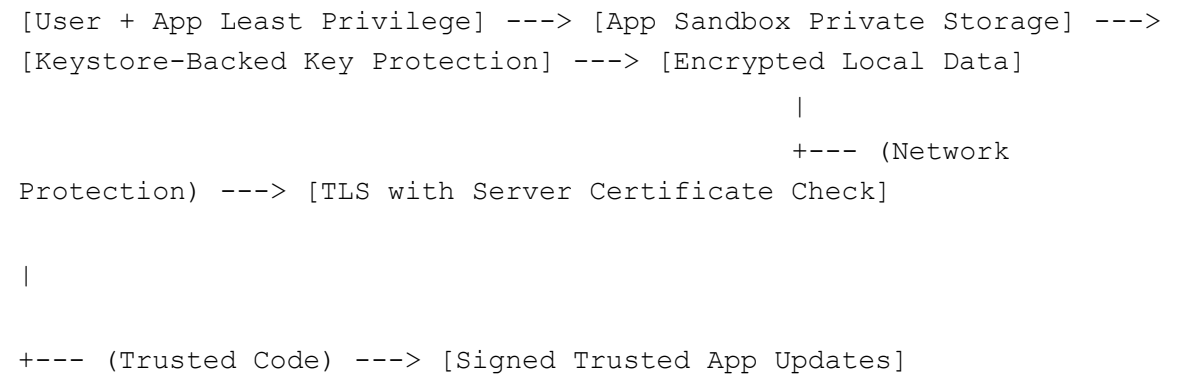
Diagram 3. Android application launch sequence





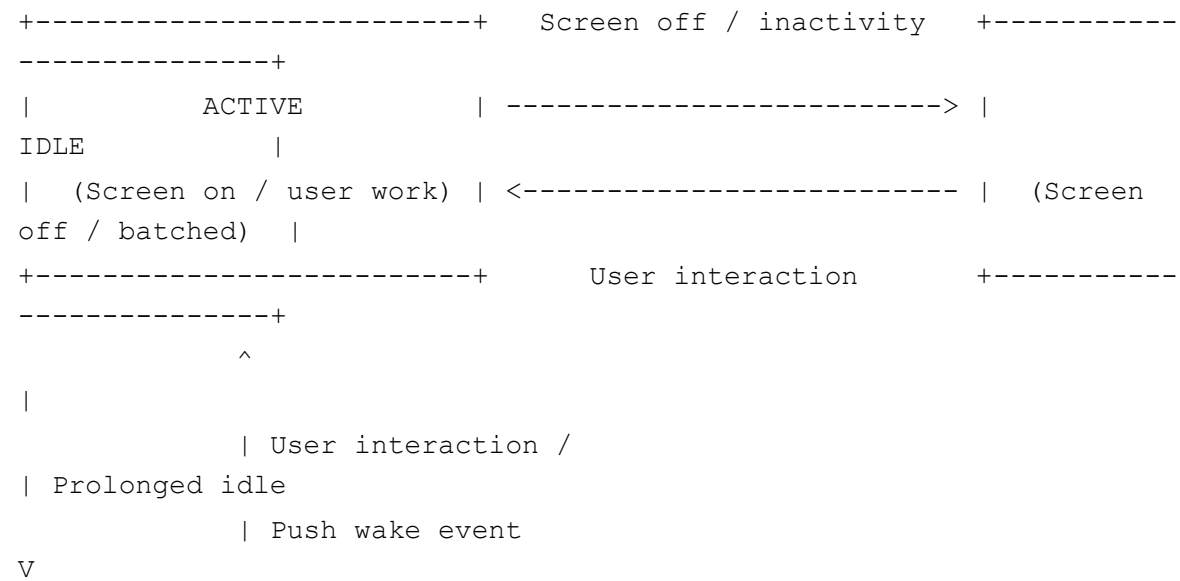
Related to Part I, Task 3: ordered steps from tapping the icon to the first rendered frame.

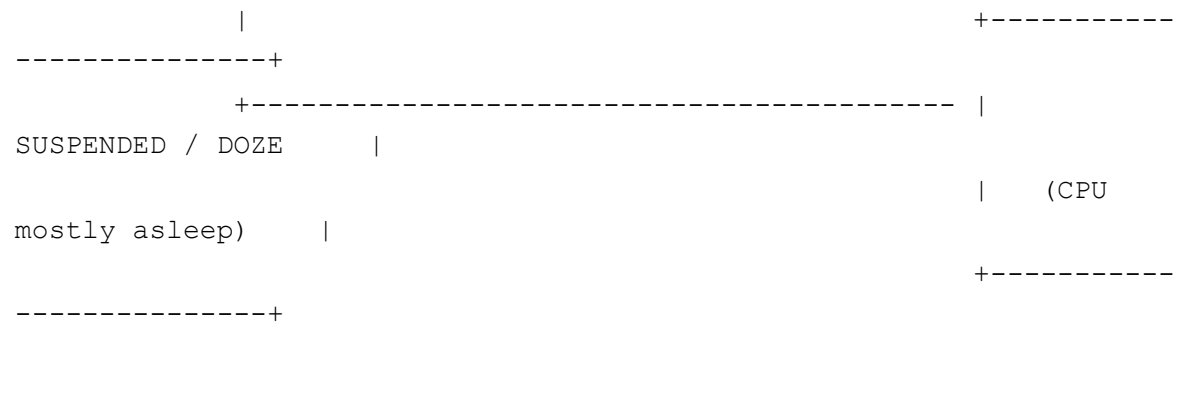
Diagram 4. SecureNotes repaired security design



Related to Part I, Task 4: repaired security controls for storage, network and updates.

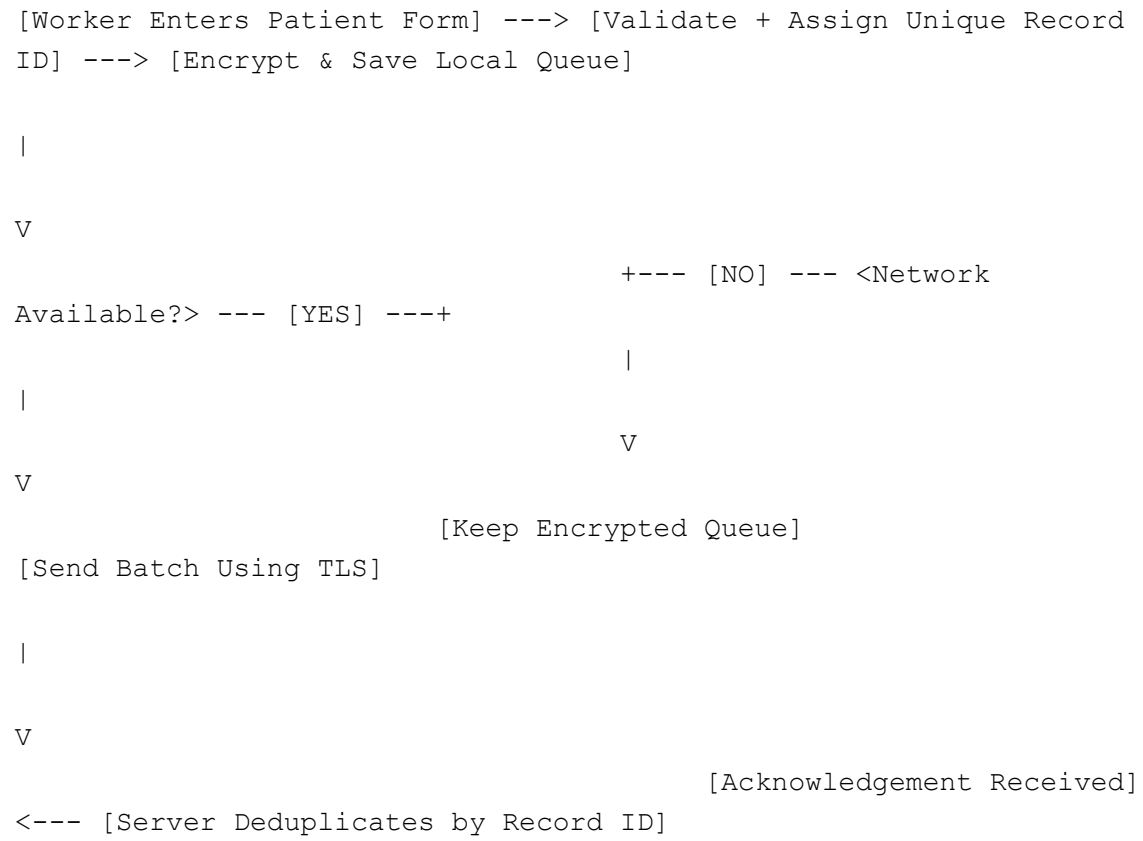
Diagram 5. Device power-state transitions





Related to Part I, Task 5: active, idle and suspended states with transition triggers.

Diagram 6. RuralCare offline capture and safe synchronisation



Related to Part I, Task 6 and Part II, Task 6: offline data capture and acknowledgement-based sync.